Name: Rhilo Sotto
RED ID: 130551574
Date: 11/1/2024

Lab 6 Report

1. What does your program do?

The program uses interrupts to scan a keypad periodically and generate a duty cycle waveform based on which button is pushed. The program runs continuously, and since there is no break statement, it will run indefinitely until power is lost.

2. What variables/registers does your program use?

The **DDRD** register is used to set the rows of the GPIO keypad to outputs

The **DDRB** register is used to set the columns of the GPIO keypad to inputs and output to the LED.

The **PORTD** register is used to set rows to logic high and isolate the row of a button pressed during iterative scanning.

The **PORTB** register is used to enable the pull-up resistor property and make the default input to the port logic high, and logic low when the keypad button is pressed. The register is also used to output either high or low to the LED, turning it on or off.

The **PINB** register is used to read the Port B input pins during iterative scanning.

The **TCCR0A** and **TCCR2A** registers set Timer 0 and 2 to CTC mode.

The **TCCR0B** and **TCCR2B** registers to set the prescaler to x1024 and x128 and start timer 0 and 2 respectively.

The **OCR0A** and **OCR2A** registers act as Timer 0's and Timer 2's max value with which the counter would reset to 0 upon reaching, effectively the fraction of the pre-scaled period that Timer 0 and 2 would elapse before resetting.

The **OCR2B** register acts as Timer 2's duty cycle for the LED PWM.

The **buttonmap** 2D array is used to map the row and column of the GPIO keypad to the volatile global short int variable **button** value associated with it.

The **TIMSK0** and **TIMSK2** registers set the corresponding timer's COMPA and COMPA and COMPB vectors respectively.

3. What is the main algorithm/logic of your program?

The main program calls the **init** function to initialize the DDRB register to set the LED on PINB5 as an output.

Then it calls the **GPIO_init** function to initialize the Ports D and B to the settings used in Lab 3, pull-up input/output respectively.

Then it calls the **timer_init** function to set Timer 0's and 2's modes to CTC which will cause the clock to reset upon reaching the value stored in OCR0A and OCR2A. TCCR0B and TCCR2B are used to set the prescaler value to x1024 and x128 and start the timers, which were chosen since the required interrupt period given my ID is 5+1ms, and the PWM frequency is 500Hz (2ms) respectively. These can be achieved with max periods of 16.384 ms and 2.048ms in CTC mode.

Global interrupts are then enabled in main.

The program enters the infinite while loop, where a switch case checks the current value of the volatile global short int variable **button**, and setting OCR2B to a corresponding duty cycle percentage (pre-calculated).

The program has three interrupt service routines.

1. **ISR (TIMER0_COMPA_vect)** is Timer0's compare register A interrupt, which occurs every 6ms, it iteratively scans the GPIO keypad and sets **button** to a value that is mapped in the 2D array, **buttonmap**
2. **ISR (TIMER2_COMPA_vect)** is Timer2's compare register A interrupt, which turns on the LED, except if the variable button has a value of 0, which represents a 0% duty cycle of not turning the LED on at all.
3. **ISR (TIMER2_COMPB_vect)** is Timer2's compare register B interrupt, which turns off the LED.

4. What external hardware connections did you make?

The GPIO keypad is connected to my microcontroller by pins PD4-PD7, and PB0-PB3, where the rows correspond to port D and columns correspond to port B. The LED is onboard, wired to PB5.

Appendix (Code)

```
/*
 * Lab6.c
 *
 * Created: 10/29/2024 1:34:31 PM
 * Author : rnsot
 */

#define F_CPU 16000000UL // 16MHz clock speed

#include <avr/io.h>
#include <avr/interrupt.h>

// REDID: 130551574
// X = 5
//		Timer interrupt period = (5 + 1) ms = 6ms
// Y = 7
// Z = 4
//		PWM frequency = (4 + 1) * 100Hz = 500Hz
//				2 ms period


// Global Variable
volatile short int button = -1;

short int buttonmap[4][4] =
{
		{1, 2, 3, 0},
		{4, 5, 6, 0},
		{7, 8, 9, 0},
		{0, 0, 0, 0}
};

void init(void) {
		DDRB |= (1 << DDRB5); // output to LED
}

void GPIO_init(void) {
		// Rows = PORTD, Columns = PORTB
		// set rows to output
		DDRD |= (1<<DDD4 | 1<<DDD5 | 1<<DDD6 | 1<<DDD7); // Port D (4-7)
		// set column to input
		DDRB &= ~(1<<DDB0 | 1<<DDB1 | 1<<DDB2 | 1<<DDB3); // Port B (0-3)
```

```
	// pull up property enabled (columns)
	PORTB |= (1<<PORTB0 | 1<<PORTB1 | 1<<PORTB2 | 1<<PORTB3);
	// rows to logic high
	PORTD |= (1<<PORTD4 | 1<<PORTD5 | 1<<PORTD6 | 1<<PORTD7);
}

void timer_init(void) {
	// FCPU = 16MHz, want 6ms and and 2ms
	// TIMER 0 - interrupt timer (6ms)
	// ps = 1, NM -> 16us = 0.016ms
	// ps = 8, NM -> 128us = 0.128ms
	// ps = 64, NM -> 1024us = 1.024ms
	// ps = 256, NM -> 4096us = 4.096ms
	// ps = 1024, NM -> 16384us = 16.384ms **

	TCCR0A |= (1 << WGM01); // CTC Mode

	OCR0A = 93; // timer period (6ms/16.384ms * 256 time steps = 93.75 time steps - 1 = 93
OCRA)

	TIMSK0 |= (1 << OCIE0A); // set ISR COMPA vector

	// TIMER 2 - PWM timer (2ms)
	// ps = 1, NM -> 16us = 0.016ms
	// ps = 8, NM -> 128us = 0.128ms
	// ps = 64, NM -> 1024us = 1.024ms
	// ps = 128, NM -> 2048us = 2.048ms **
	// ps = 256, NM -> 4096us = 4.096ms
	// ps = 1024, NM -> 16384us = 16.384ms

	TCCR2A |= (1 << WGM21); // CTC Mode

	OCR2A = 249; // timer period (2ms/2.048ms * 256 time steps = 250 time steps - 1 = 249
OCRA)
	OCR2B = 124; // duty cycle

	TIMSK2 |= (1 << OCIE2A) | (1 << OCIE2B); // set ISR COMPA and COMPB vector

	TCCR0B |= (1 << CS02) | (1 << CS00); // Pre-scaler set to 1024, STARTS timer0
	TCCR2B |= (1 << CS22) | (1 << CS20); // Pre-scaler set to 128, STARTS timer2
}


int main(void) {
```

```
        init(); // initialize inputs and outputs
        GPIO_init(); // initialize GPIO keypad
        timer_init(); // timer settings

        sei(); // enable global interrupts

    while (1) {
                switch(button) {
                        case 0: // 0% duty cycle
                                OCR2B = 0; // 0
                                break;
                        case 1: // 10% duty cycle
                                OCR2B = 24; // 24
                                break;
                        case 2: // 20% duty cycle
                                OCR2B = 49; // 49
                                break;
                        case 3: // 30% duty cycle
                                OCR2B = 74; // 74
                                break;
                        case 4: // 40% duty cycle
                                OCR2B = 99; // 99
                                break;
                        case 5: // 50% duty cycle
                                OCR2B = 124; // 124
                                break;
                        case 6: // 60% duty cycle
                                OCR2B = 149; // 149
                                break;
                        case 7: // 70% duty cycle
                                OCR2B = 174; // 174
                                break;
                        case 8: // 80% duty cycle
                                OCR2B = 199; // 199
                                break;
                        case 9: // 90% duty cycle
                                OCR2B = 224; // 224
                                break;
                        default:
                                break;
                }
    }
}
```

```
ISR (TIMER0_COMPA_vect) { // Timer0 compare register A interrupt
	/* iterative scan of GPIO keypad */
	for (char i = 4; i < 8; ++i) {
		// set row[i] low
		PORTD &= ~(1<<i);
		for (char j = 0; j < 4; ++j) {
			// check column[j] if low
			if ( !(PINB & (1<<j)) ) { // first transition low (on)
				button = buttonmap[i-4][j]; // set button to mapped value
			}
		}
		// set row[i] high
		PORTD |= (1<<i);
	}
}

ISR (TIMER2_COMPA_vect) { // Timer2 compare register A interrupt
	if (button != 0) PORTB |= (1 << PORTB5); // HIGH, LED ON
}

ISR (TIMER2_COMPB_vect) { // Timer2 compare register B interrupt
	PORTB &= ~(1 << PORTB5); // LOW, LED OFF
}
```